

Wallet Ledger Service Implementation Plan

Summary

Implement a raw Node.js HTTP server with 4 API endpoints for a ledger-based wallet service. The critical requirement is concurrency safety using PostgreSQL advisory locks to prevent overdrafts during simultaneous purchases.

Assessment Requirements Checklist

- ✔ Ledger-based balance (sum of transactions, no balance column)
- ✔ Concurrency safety with PostgreSQL advisory locks
- ✔ Price snapshot at purchase time (stored in DebitDetail.amount)
- ✔ Database indexes (already in schema)
- ✔ Idempotency design note (README, not implemented)

API Endpoints

Method	Path	Description	Response
GET	/api/items	List 4 hardcoded items	[{id, name, price}]
GET	/api/balance	Get user balance	{balance: number}
POST	/api/credits	Add credits	{balance: number}
POST	/api/purchases	Purchase item	204 No Content

All endpoints require **x-user-id: <UUID>** header (400 if missing/invalid)

File Structure

```
apps/wallet/src/
├── index.ts           # Entry point - starts HTTP server
├── server.ts         # WalletServer class with routing
├── handlers/
│   ├── items.ts      # GET /api/items
│   ├── balance.ts    # GET /api/balance
│   ├── credits.ts    # POST /api/credits
│   └── purchases.ts  # POST /api/purchases
├── services/
│   ├── items.ts      # Hardcoded 4-item catalog
│   └── ledger.ts     # Balance calc + advisory lock purchase
└── utils/
    ├── http.ts       # Request/response helpers
    └── validation.ts  # UUID validation
```

Critical Implementation Details

1. Hardcoded Items (from seed data)

Select 4 items from `packages/seed/src/items/index.ts`:

```
export const CATALOG_ITEMS = [
  { id: "33456444-29af-4484-b5d1-af61d06ef889", name: "iPhone 13 Pro", price:
```

```
109999 },
  { id: "b8c887ee-d5c6-4815-98fe-7dd102b80d73", name: "iPhone X", price: 89999 },
  { id: "f71bc01c-b3c2-464c-a54c-a857100ef171", name: "Apple AirPods Max Silver", price: 54999 },
  { id: "372d8cd7-2c2b-4d7f-b366-5af2d63509f8", name: "Apple Watch Series 4 Gold", price: 34999 }
];
```

2. Balance Calculation

```
const result = await prisma.ledgerEntry.aggregate({
  where: { userId, status: "COMPLETED" },
  _sum: { amount: true }
});
return result._sum.amount ?? 0n;
```

- Credits stored as **positive** amounts
- Debits stored as **negative** amounts
- Sum gives balance directly

3. Concurrency-Safe Purchase Flow (Advisory Locks)

```
return await prisma.$transaction(async (tx) => {
  // 1. Acquire per-user advisory lock
  const lockKey = uuidToLockKey(userId);
  await tx.$executeRawUnsafe(`SELECT pg_advisory_xact_lock($1)`, lockKey);

  // 2. Calculate balance WITHIN transaction
  const balance = await calculateBalance(tx, userId);

  // 3. Check sufficient funds
  if (balance < item.price) {
    return { success: false }; // 409 Insufficient balance
  }

  // 4. Create debit entry with price snapshot
  await tx.ledgerEntry.create({
    data: {
      userId,
      entryType: "DEBIT",
      amount: -BigInt(item.price), // Negative for debit
      debitDetail: {
        create: {
          itemId: item.id,
          itemName: item.name,
          amount: BigInt(item.price) // Price snapshot
        }
      }
    }
  });

  return { success: true };
  // Lock auto-released on commit
});
```

Why advisory locks?

- Serializes concurrent requests per-user

- Other users not blocked
- Works for new users (no existing rows needed)
- Transaction-scoped (auto-release on commit/rollback)

4. HTTP Server (Raw Node.js)

Pattern from [apps/pdf-pipeline.md](#):

```
const server = http.createServer(async (req, res) => {
  const userId = extractUserId(req);
  if (!userId) {
    return sendError(res, 400, "x-user-id header required");
  }

  // Route matching
  if (req.url === "/api/items" && req.method === "GET") {
    return handleGetItems(req, res);
  }
  // ... other routes
});
```

Implementation Sequence

1. **Utils** - [http.ts](#), [validation.ts](#) (request parsing, UUID validation)
2. **Services** - [items.ts](#), [ledger.ts](#) (catalog, balance, advisory lock purchase)
3. **Handlers** - [items.ts](#), [balance.ts](#), [credits.ts](#), [purchases.ts](#)
4. **Server** - [server.ts](#) (routing), [index.ts](#) (entry point)
5. **README** - Run locally, migrations, concurrency explanation, idempotency note

README Sections

Concurrency Explanation (for README)

The purchase endpoint uses PostgreSQL advisory locks to ensure atomic balance checks and debits. When concurrent purchase requests arrive for the same user, the first acquires a lock keyed to the user's ID, blocking subsequent requests until completion. Each request recalculates balance within its transaction, guaranteeing balance cannot go negative. Different users can purchase concurrently without blocking.

Idempotency Note (5-10 sentences)

For production, idempotency would use the existing IdempotencyRecord model. Clients include `x-idempotency-key` header with mutating requests. Before processing, the server checks for an existing (key, userId) record. If found, the cached response replays immediately. If not, a record is created with "PENDING" state, then updated with the final response. Records include endpoint, status code, and response body. For purchases, the record links to the LedgerEntry via ledgerEntryId. Records expire after 24 hours for cleanup.

Error Handling

Condition	Status	Response
Missing/invalid x-user-id	400	<code>{"error": "x-user-id header required"}</code>
Invalid JSON body	400	<code>{"error": "Invalid JSON body"}</code>
amount <= 0 or not integer	400	<code>{"error": "amount must be a positive integer"}</code>
Item not found	404	<code>{"error": "Item not found"}</code>

Condition	Status	Response
Insufficient balance	409	<code>{"error": "Insufficient balance"}</code>

Verification

```
# 1. Get items
curl http://localhost:3000/api/items -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000"

# 2. Check balance (new user = 0)
curl http://localhost:3000/api/balance -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000"

# 3. Add credits
curl -X POST http://localhost:3000/api/credits \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000" \
  -H "Content-Type: application/json" \
  -d '{"amount": 200000}'

# 4. Purchase item
curl -X POST http://localhost:3000/api/purchases \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000" \
  -H "Content-Type: application/json" \
  -d '{"itemId": "33456444-29af-4484-b5d1-af61d06ef889"}'

# 5. Concurrent purchase stress test
for i in {1..5}; do
  curl -X POST http://localhost:3000/api/purchases \
    -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000" \
    -H "Content-Type: application/json" \
    -d '{"itemId": "33456444-29af-4484-b5d1-af61d06ef889"}' &
done
wait
```

Critical Files

- `apps/wallet/src/index.ts` - Entry point (needs implementation)
- `packages/db/src/db.ts` - DbService with PrismaPg adapter
- `packages/seed/src/items/index.ts` - Source of 4 catalog items
- `packages/db/prisma/schema/*.prisma` - LedgerEntry, CreditDetail, DebitDetail models